## Topic 7: ARC and Memory Management - Complete Code

### Task 1 (a): Create the retain cycle - strong tenant

```
class Person {
   let name: String
   var apartment: Apartment?

   init(name: String) {
      self.name = name
      print("\(name) is created")
   }

   deinit {
      print("\(name) is deinitialized")
   }
}

class Apartment {
   let unit: String
   var tenant: Person?

   init(unit: String) {
      self.unit = unit
      print("Apartment \(unit) is created")
   }

   deinit {
      print("Apartment \(unit) is deinitialized")
   }
}

var person: Person? = Person(name: "Asha")
var apartment: Apartment? = Apartment(unit: "4B")

person?.apartment = apartment
apartment?.tenant = person

person = nil
apartment = nil
print("Both variables are nil, but no deinit message appeared")
```

**Output**

```
Asha is created
Apartment 4B is created
Both variables are nil, but no deinit message appeared
```

**Task 1 (b): Break the cycle - weak tenant**

```
class Person {
    let name: String
    var apartment: Apartment?

    init(name: String) {
        self.name = name
        print("\(name) is created")
    }

    deinit {
        print("\(name) is deinitialized")
    }
}

class Apartment {
    let unit: String
    weak var tenant: Person?

    init(unit: String) {
        self.unit = unit
        print("Apartment \(unit) is created")
    }

    deinit {
        print("Apartment \(unit) is deinitialized")
    }
}

var person: Person? = Person(name: "Asha")
var apartment: Apartment? = Apartment(unit: "4B")

person?.apartment = apartment
apartment?.tenant = person

person = nil
print("Person variable is nil")
apartment = nil
```

print("Apartment variable is nil")

**Output**

```
Asha is created
Apartment 4B is created
Asha is deinitialized
Person variable is nil
Apartment 4B is deinitialized
Apartment variable is nil
```

**Task 2 (a): Closure capture cycle - strong self**

```
class Downloader {
    let name: String
    var onFinish: (() -> Void)?

    init(name: String) {
        self.name = name
    }

    deinit {
        print("\(name) downloader is deinitialized")
    }

    func start() {
        onFinish = {
            print("Finished \(self.name)")
        }
    }
}

var downloader: Downloader? = Downloader(name: "photo.zip")
downloader?.start()
downloader?.onFinish?()

downloader = nil
print("Downloader set to nil, but no deinit message appeared")
```

**Output**

```
Finished photo.zip
Downloader set to nil, but no deinit message appeared
```

**Task 2 (b): Fixed with a capture list**

```
class Downloader {
   let name: String
   var onFinish: (() -> Void)?

   init(name: String) {
      self.name = name
   }

   deinit {
      print("\(name) downloader is deinitialized")
   }

   func start() {
      onFinish = { [weak self] in
         guard let self = self else { return }
         print("Finished \(self.name)")
      }
   }
}

var downloader: Downloader? = Downloader(name: "photo.zip")
downloader?.start()
downloader?.onFinish?()

downloader = nil
print("Downloader set to nil")
```

**Output**

```
Finished photo.zip
photo.zip downloader is deinitialized
Downloader set to nil
```

**Task 3: When does the object die?**

```
class Box {
   deinit {
      print("Box gone")
   }
}

var a: Box? = Box()
var b = a
var c = a
```

```
a = nil
b = nil
print("before c")
c = nil
print("end")
```

**Output**

```
before c
Box gone
end
```